



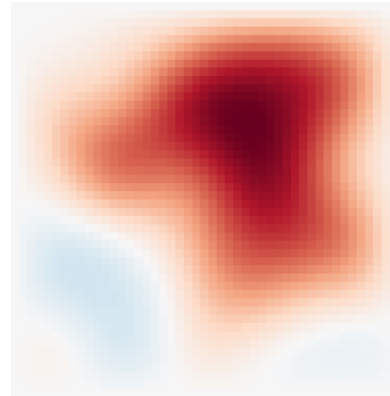
ML FOR SCIENCE · LECTURE 3

PDE and Numerical Fundamentals

How was Lecture 1 data generated?

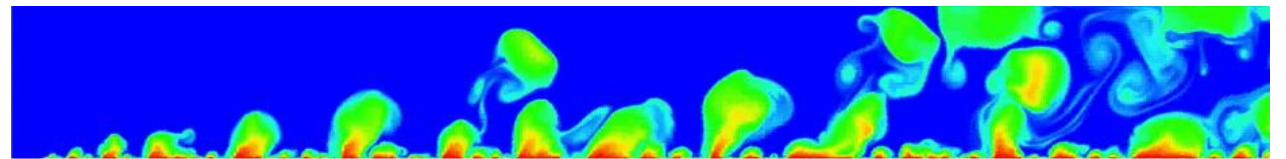
- **Heat equation** data: computed **from a formula**
 - a sum of decaying sine modes on a 48×48 grid
 - frames $\Delta t = 5 \times 10^{-4}$ apart, $\alpha = 1$, $u = 0$ on the walls
- **BubbleML** data: computed **by a solver** (Flash-X)
 - solver time step $\leq 4 \times 10^{-4}$; 201 frames saved over $t = 0$ to 200
 - so **at least 2,500 solver steps** between saved frames
- Most SciML training data comes from solvers, and *the solver's choices end up in the data*
- Today's three questions
 - What equation is being solved?
 - How does a solver turn it into arrays?
 - How accurate is the result?

Heat equation: a formula (Lecture 1 notebook, 48×48)



$u = 0$ on the walls; each sine mode decays at its own rate

BubbleML: a solver (Flash-X), temperature

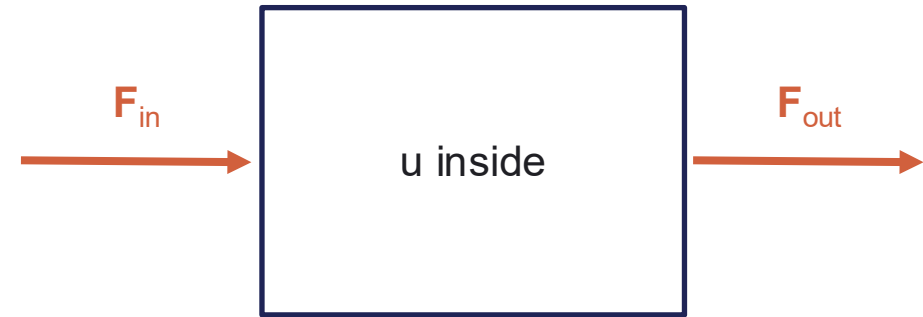


heater along the bottom wall; bubbles grow, detach and rise

Every training set in this course is the output of a numerical method.

PDEs as conservation laws

- Take a small cell of the domain. The amount of a conserved quantity inside it (heat, mass, momentum) changes *only* by what flows across its walls
- The flow per unit area per unit time is the **flux** $\mathbf{F}$
- Shrink the cell to a point: a **conservation law**
- Conservation alone does **not** close the equation. We also need a **flux law**: how $\mathbf{F}$ depends on u
- Fourier's law: heat flows from hot to cold, at a rate proportional to the temperature gradient. This gives the heat equation
 - α : diffusivity (units m^2/s); ∇u : gradient of u ; $\nabla \cdot \mathbf{F}$: divergence (net outflow per unit volume)



$$\frac{d}{dt} \int_{\text{cell}} u \, dx = F_{\text{in}} - F_{\text{out}}$$

$$\frac{\partial u}{\partial t} + \nabla \cdot \mathbf{F} = 0$$

$$\mathbf{F} = -\alpha \nabla u \quad \implies \quad \frac{\partial u}{\partial t} = \alpha \nabla^2 u$$

PDE = a conservation law + a flux law. Different physics gives us a different flux law.

The big three: mass, momentum and energy

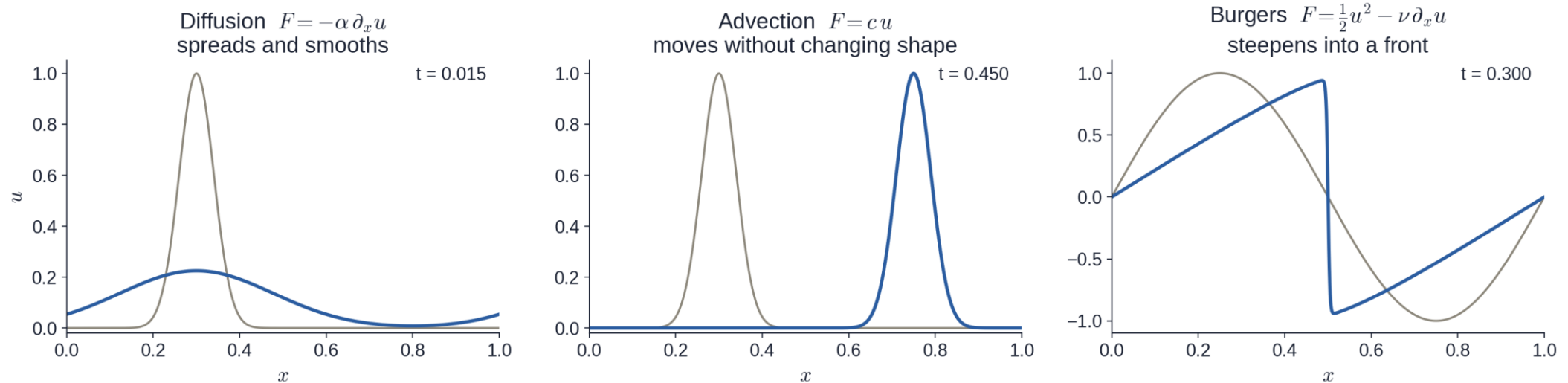
conserved quantity	flux	conservation law
Mass density ρ	carried by the flow: $\rho \mathbf{u}$	$\frac{\partial \rho}{\partial t} + \nabla \cdot (\rho \mathbf{u}) = 0 \xrightarrow{\rho \text{ constant}} \nabla \cdot \mathbf{u} = 0$
Momentum $\rho \mathbf{u}$ (Newton's second law)	carried momentum $\rho \mathbf{u} \otimes \mathbf{u}$, pressure p , viscous stress τ	$\frac{\partial(\rho \mathbf{u})}{\partial t} + \nabla \cdot (\rho \mathbf{u} \otimes \mathbf{u} + p \mathbf{I} - \tau) = \rho \mathbf{g}$
Energy thermal energy $\rho c_p T$	carried by the flow, conduction $-k \nabla T$ (Fourier's law)	$\frac{\partial(\rho c_p T)}{\partial t} + \nabla \cdot (\rho c_p T \mathbf{u} - k \nabla T) = 0$

- Together these are the Navier-Stokes equations for a compressible, heat-conducting fluid. With constant density, mass conservation becomes $\nabla \cdot \mathbf{u} = 0$
- Other conserved quantities: chemical species (reaction-diffusion), electric charge (electromagnetics)

BubbleML is these three laws plus an equation that moves the bubble surface.

Same idea, different flux

- **Diffusion**, $F = -\alpha \partial u / \partial x$: spreads and smooths; sharp features disappear first
- **Advection**, $F = c u$ (material carried at speed c): moves the profile without changing its shape
- **Burgers**, $F = \frac{1}{2}u^2 - \nu \partial u / \partial x$: the speed is u itself, so high parts overtake low parts and **a steep front forms**; the viscosity ν keeps it from becoming a true jump
- **Navier–Stokes**: conservation of momentum and mass, with advection, pressure and viscous fluxes



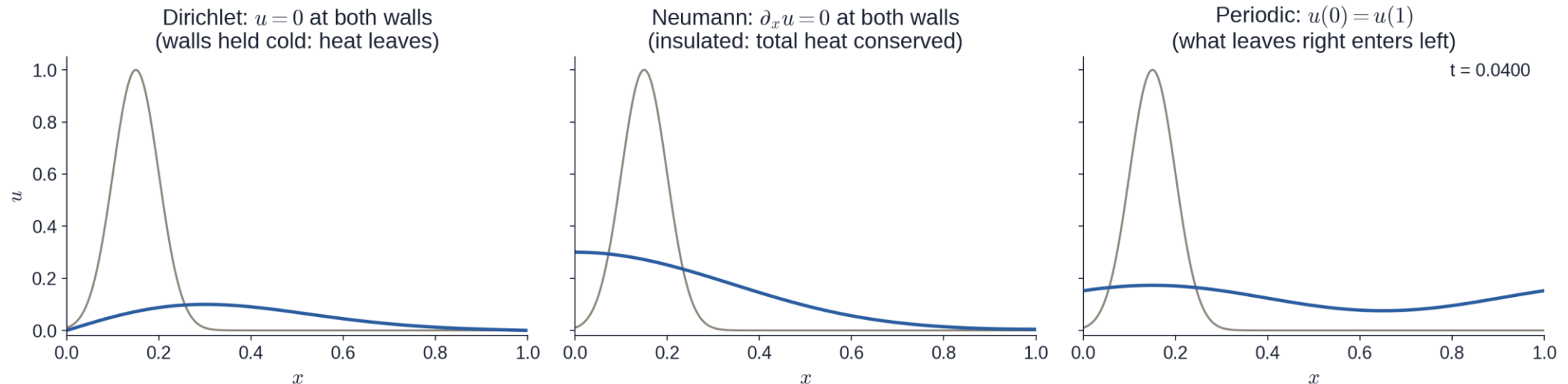
Every PDE in this course combines these few behaviors.

Initial and boundary conditions

- A PDE alone has *infinitely many* solutions. To pick one we also need
 - an **initial condition** $u(x, 0) = u_0(x)$
 - a **boundary condition** on every wall
- Common boundary conditions
 - **Dirichlet**: fixed value (the BubbleML heater wall held at T_{wall})
 - **Neumann**: fixed flux; zero flux means an insulated wall
 - **Periodic**: the domain wraps around (KS and many ML benchmarks)

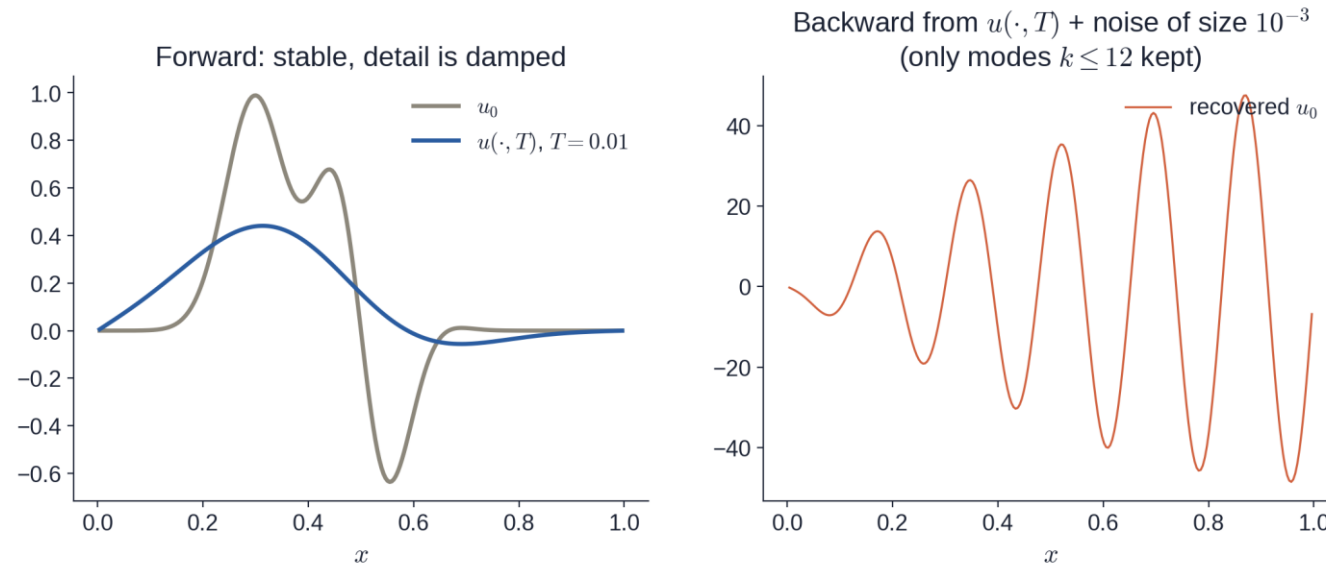
$$u(x, 0) = u_0(x), \quad u|_{\text{wall}} = g \quad \text{or} \quad \frac{\partial u}{\partial n}|_{\text{wall}} = h$$

PDE + initial condition + boundary conditions define the solution operator $\mathcal{G}$ from Lecture 1



Well-posed problems

- **Hadamard** (1902): a problem is **well posed** if a solution exists, is unique, and depends continuously on the data (small change in $u_0 \rightarrow$ small change in u)
- **Forward heat equation**: well posed. Each Fourier mode is multiplied by $e^{-\alpha\pi^2k^2t} \leq 1$
- **Backward heat equation** (recover u_0 from u at time T): **not well posed**. Each mode is multiplied by $e^{+\alpha\pi^2k^2T}$
 - mode $k = 10$, $T = 0.01$: amplified 1.9×10^4 times
 - noise of size 10^{-3} in $u(\cdot, T)$ gives a "recovered" u_0 of size **49**
- This is the inverse problem from Lecture 1. It needs extra assumptions (regularization, priors) to have a stable answer



Linear vs. nonlinear PDEs

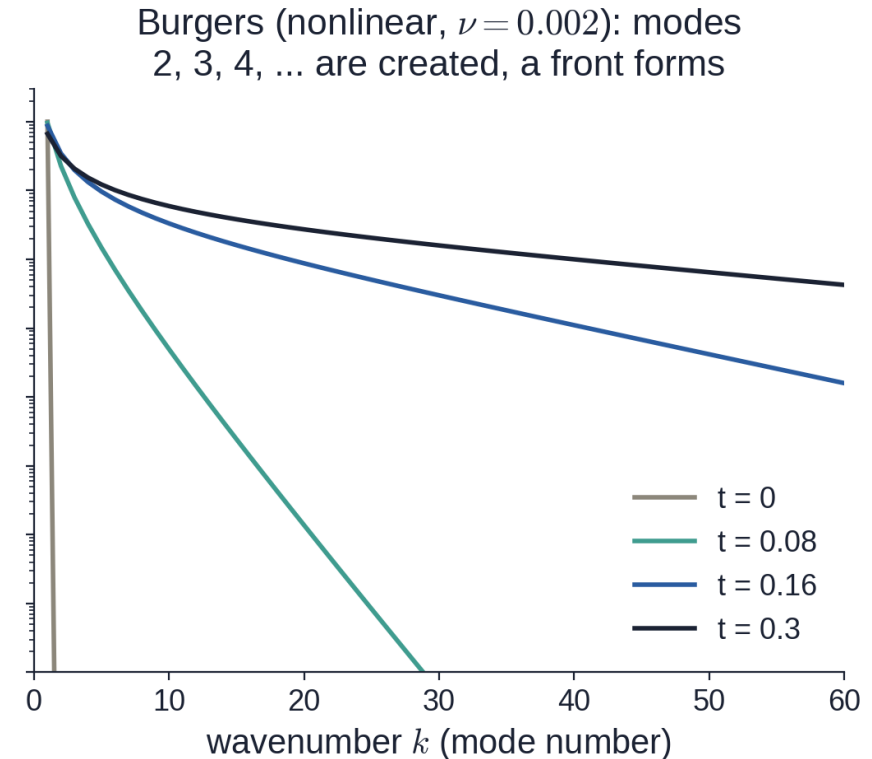
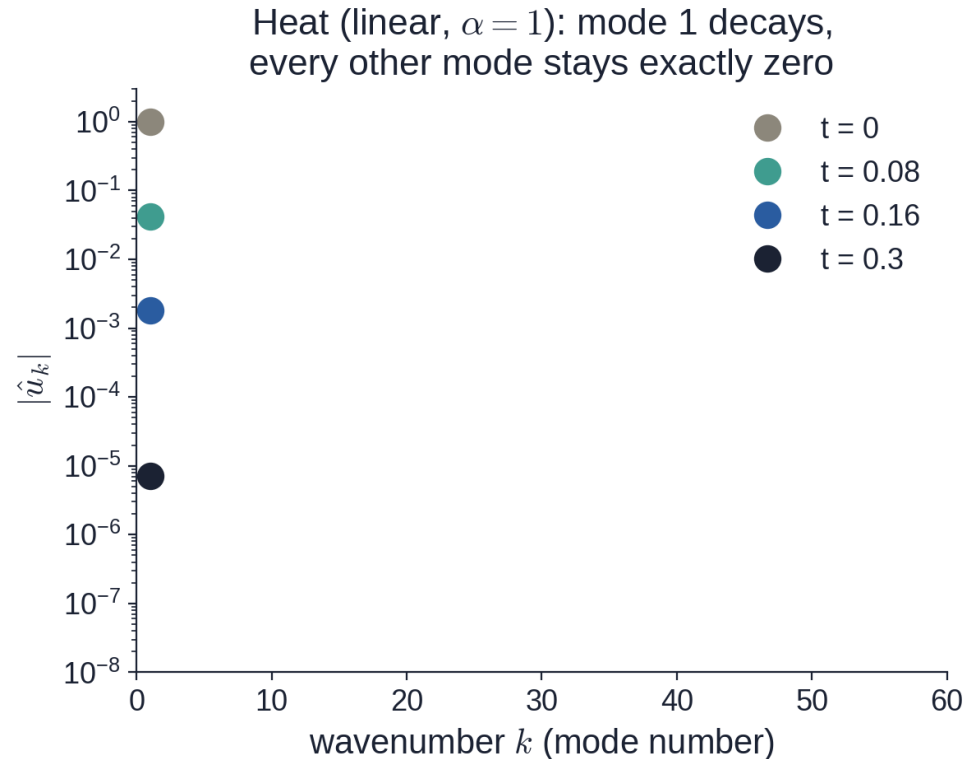
- **Linear**: if u and v are solutions, so is $a \cdot u + b \cdot v$ (superposition)
 - heat, advection, waves, Poisson
 - consequence: each Fourier mode evolves *on its own*, at its own rate
 - the Lecture 1 heat data is exactly this: independently decaying sine modes
- **Nonlinear**: superposition **fails**
 - Burgers, Navier-Stokes, KS
 - the term $u \partial u / \partial x$ multiplies modes: modes k and k' *create* $k + k'$ and $k - k'$
 - this is how fronts and fine-scale structure form
- Preview: an FNO layer (Lecture 11) is a linear map on Fourier modes followed by a pointwise nonlinearity

$$\frac{d\hat{u}_k}{dt} = -\alpha (2\pi k)^2 \hat{u}_k$$

heat equation on a periodic domain $[0, 1]$; $\hat{u}_k$ is the coefficient of mode k (wavenumber $2\pi k$)

Linear: modes evolve independently. **Nonlinear**: modes create new modes.

Same start, one mode: linear keeps it, nonlinear spreads it



- Both start from $u_0 = \sin 2\pi x$, which is mode 1
- Heat: mode 1 decays and every other mode stays *exactly zero*. Burgers: modes 2, 3, 4, ... appear and a front forms

One heat step is a linear map. Burgers and BubbleML are not.

Three types of second-order PDE

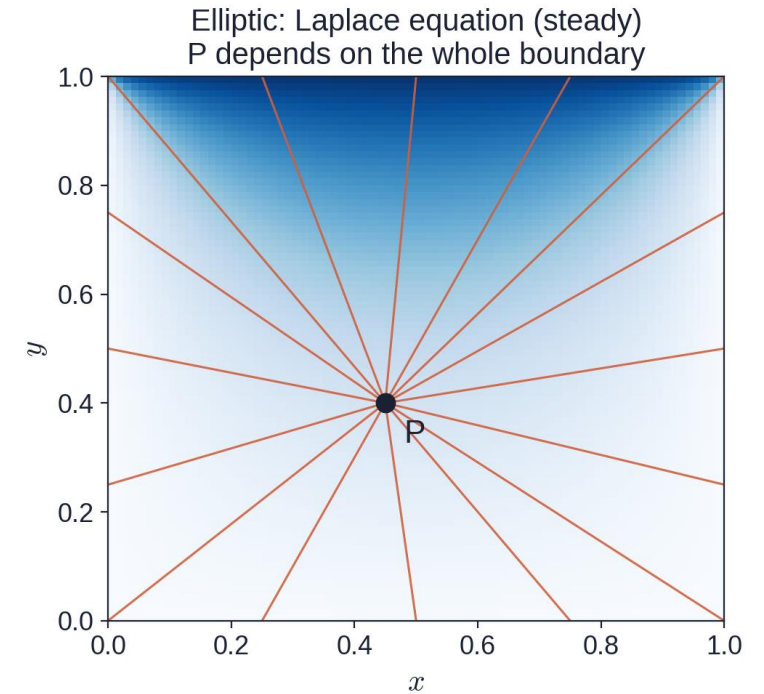
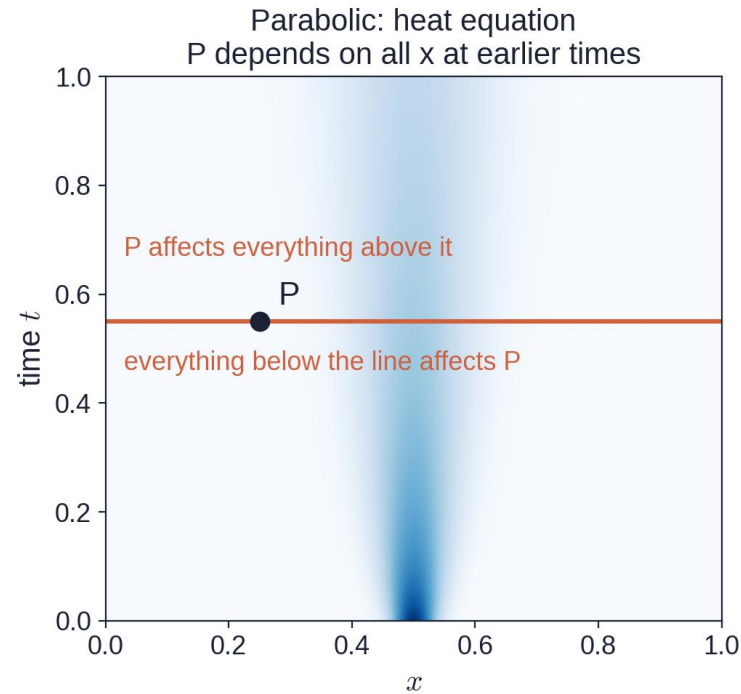
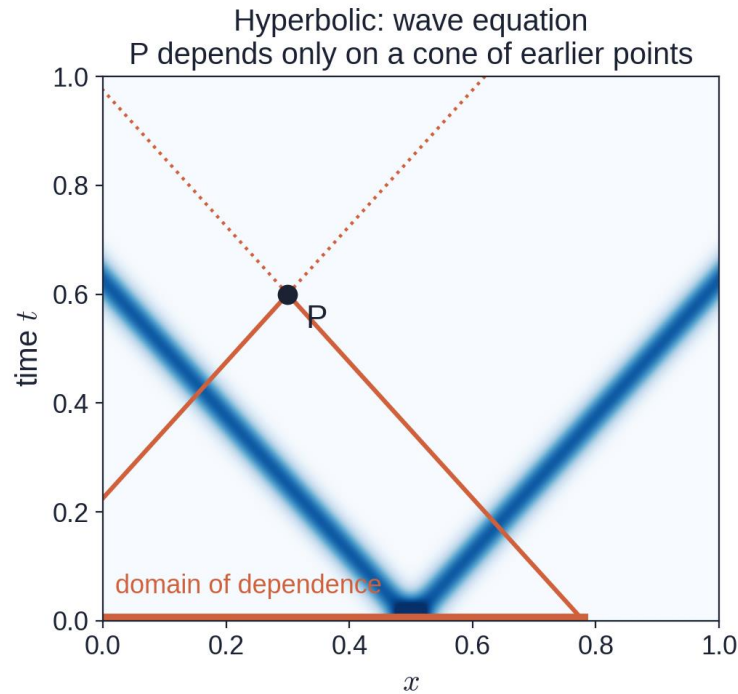
$$A u_{xx} + B u_{xy} + C u_{yy} + (\text{lower-order terms}) = 0$$

The sign of $\mathbf{B^2 - 4AC}$ dictates the type; the names come from the conic sections $Ax^2 + Bxy + Cy^2 = 1$. It counts the real directions along which information travels: *two, one or none*

type	example	information travels	data needed for a well-posed problem
Hyperbolic waves, advection	$u_{tt} - c^2 u_{xx} = 0$ $B^2 - 4AC = 4c^2 > 0$	along two lines, $x \pm ct$, at a <i>finite speed</i>	initial u and $\partial u / \partial t$; boundary values only where information enters (inflow)
Parabolic diffusion, heat	$u_t - \alpha u_{xx} = 0$ $B^2 - 4AC = 0$	along $t = \text{const}$: <i>instantly</i> to all x , but only forward in time	initial u , plus boundary values on every wall
Elliptic Poisson, Laplace, pressure	$u_{xx} + u_{yy} = 0$ $B^2 - 4AC = -4 < 0$	no characteristic direction: every point depends on the entire boundary	boundary values on every wall; <i>no initial condition</i>

The type tells you how far information travels, and what data a model must be given.

How information travels: domains of dependence



Orange: the region that can influence the point P. For a model that predicts one frame ahead, this region over one Δt must fit inside its receptive field

Receptive field needed per step: hyperbolic $c \cdot \Delta t$; parabolic a few $\sqrt{(\alpha \Delta t)}$; **elliptic: the entire domain.**

Steady problems: Poisson and Darcy

- Steady problems have no time derivative: the solution is in balance with its sources and boundaries

- Poisson**: the simplest steady problem

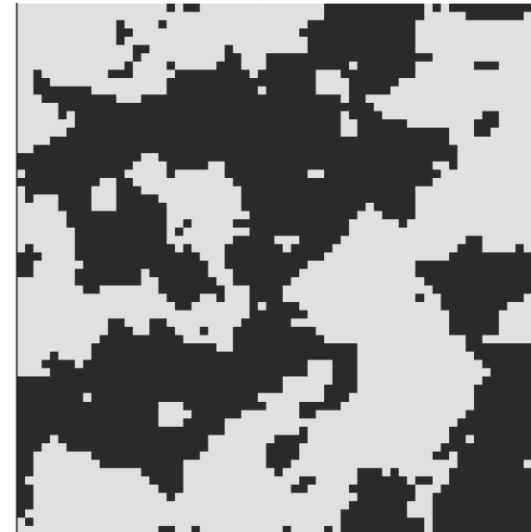
$$-\nabla^2 u = f$$

- pressure in incompressible flow: the BubbleML solver solves one at every step
- steady heat conduction, electrostatics
- Darcy flow**: Poisson with a coefficient $a(x)$ that varies in space; flow through a porous medium

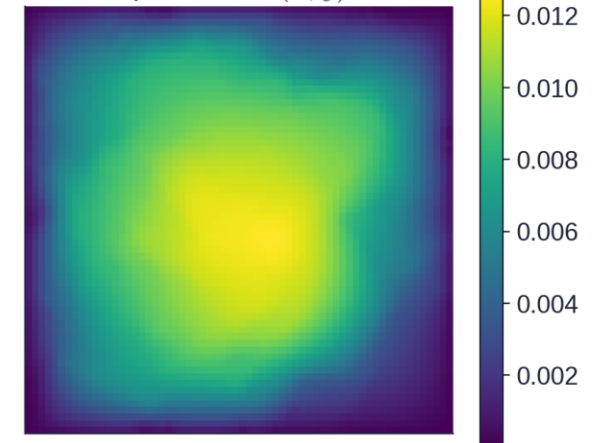
$$-\nabla \cdot (a(\mathbf{x}) \nabla u) = f$$

- u : pressure; a : permeability; f : source (here $f = 1$, $u = 0$ on the walls)
- the standard operator-learning benchmark: learn $a \mapsto u$, *function to function map*

coefficient $a(x, y)$: 12 (dark), 3 (light)



pressure $u(x, y)$



Operator learning maps an input function to an output function.

Dimensionless numbers decide which behavior wins

Temperature

$$\frac{\partial T}{\partial t} + U \frac{\partial T}{\partial x} = \alpha \frac{\partial^2 T}{\partial x^2} \quad \Rightarrow \quad \text{Pe} = \frac{U\ell}{\alpha} \quad \alpha: \text{thermal diffusivity}$$

Velocity

$$\frac{\partial u}{\partial t} + u \frac{\partial u}{\partial x} = \nu \frac{\partial^2 u}{\partial x^2} \quad \Rightarrow \quad \text{Re} = \frac{U\ell}{\nu} \quad \nu \text{ (nu): kinematic viscosity, the diffusivity of momentum (1D momentum equation)}$$

- **Péclet number** $\text{Pe} = U\ell/\alpha$: advection vs. diffusion over a length ℓ .
 - $\text{Pe} \gg 1$: fronts stay sharp
 - $\text{Pe} \ll 1$: smears
- **Reynolds number** $\text{Re} = U\ell/\nu$: the same comparison for momentum (inertia vs. viscosity)
- Both share U and ℓ , so $\text{Pe} / \text{Re} = \nu / \alpha = \text{Prandtl number } \text{Pr}$

A new Re or Pe at test time can mean **new behavior**.

Same equations, different Reynolds number

Re = 20: steady wake behind the cylinder



Re = 200: periodic vortex shedding (von Kármán street), $St \approx 0.2$



- Flow past a cylinder: Navier-Stokes equations, with $Re = UD/\nu$
 - U : inflow speed; D : cylinder diameter; ν : viscosity
- **Re = 20**: a steady wake. Solution does not change with time
- **Re $\approx$ 47**: the steady wake becomes unstable
- **Re = 200**: vortices shed periodically, alternating sides (the von Kármán street), at a frequency set by $St = fD/U \approx 0.2$
- A model trained only on steady wakes has *never seen* a time-periodic solution

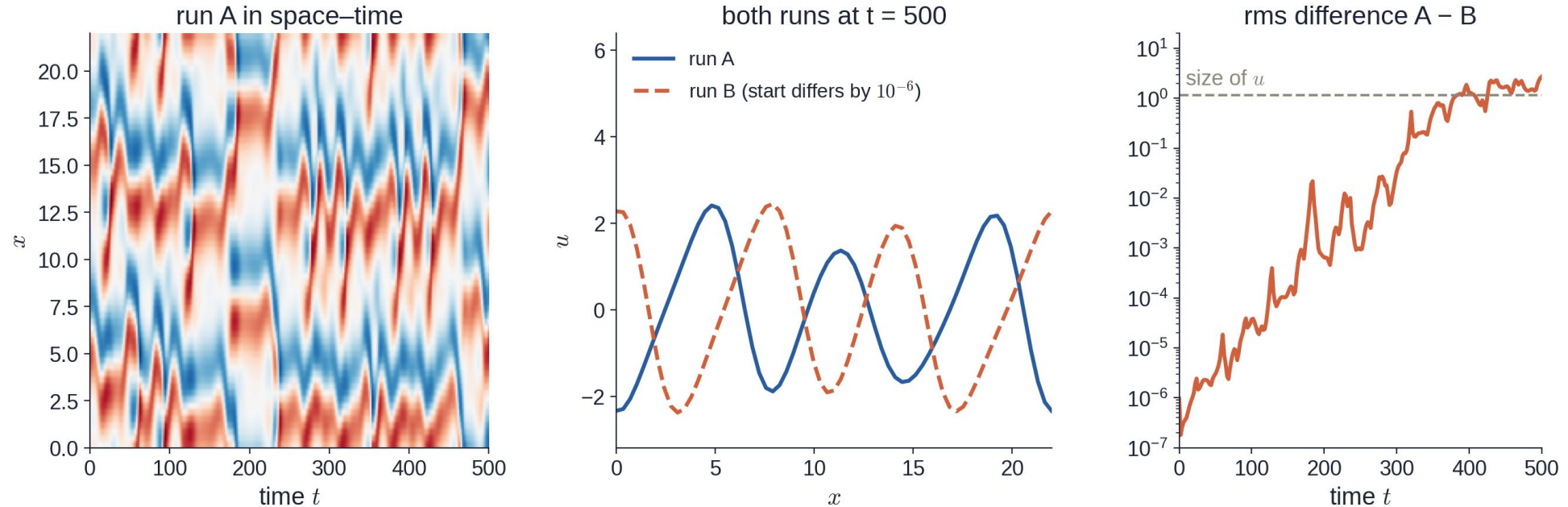
Crossing $Re \approx 47$ changes the solution.

Chaos: the Kuramoto–Sivashinsky equation

$$\frac{\partial u}{\partial t} + u \frac{\partial u}{\partial x} + \frac{\partial^2 u}{\partial x^2} + \frac{\partial^4 u}{\partial x^4} = 0$$

- The same recipe with one twist
 - $+\partial^2 u/\partial x^2$ on the left is negative diffusion: **long waves grow**
 - $\partial^4 u/\partial x^4$ damps short waves strongly; $u \partial u/\partial x$ moves energy between them
- A mode with wavenumber k grows at rate $k^2 - k^4$. On a periodic domain of length 22: modes 1-3 grow (0.075, 0.22, 0.20), mode 4 decays (-0.40)
- Energy enters at large scales and leaves at small scales. The solution *never settles*
- The standard 1D chaotic benchmark for rollout. The 2D analogue is Kolmogorov flow (Navier-Stokes with sinusoidal forcing)

Chaos: nearby states separate exponentially



- Two runs that start 10^{-6} apart separate at about 0.033 per time unit and reach the size of u after roughly **400** timesteps

Even a perfect surrogate cannot match a chaotic system pointwise over a long rollout. Compare statistics.

BubbleML equations

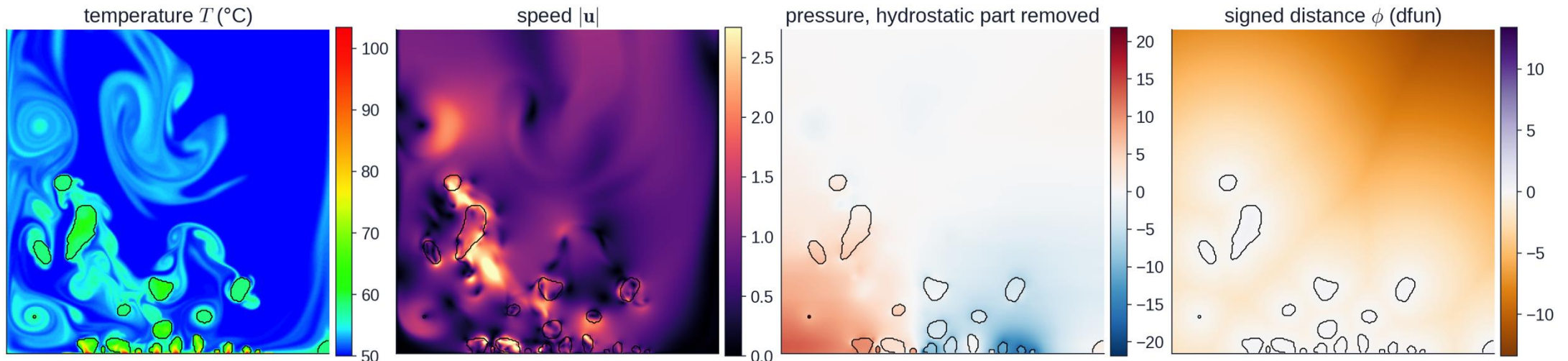
$$\frac{\partial \mathbf{u}}{\partial t} + (\mathbf{u} \cdot \nabla) \mathbf{u} = -\nabla p + \frac{1}{\text{Re}} \nabla^2 \mathbf{u} + \mathbf{g} + \text{surface tension}$$

$$\nabla \cdot \mathbf{u} = 0$$

$$\frac{\partial T}{\partial t} + \mathbf{u} \cdot \nabla T = \frac{1}{\text{Pe}} \nabla^2 T$$

$$\frac{\partial \phi}{\partial t} + \mathbf{u} \cdot \nabla \phi = 0$$

- Momentum: advection $(\mathbf{u} \cdot \nabla) \mathbf{u}$, pressure $-\nabla p$, viscous diffusion $(1/\text{Re}) \nabla^2 \mathbf{u}$, gravity, surface tension
- Mass, $\nabla \cdot \mathbf{u} = 0$ (incompressible): enforced by a Poisson solve for p at every step, so pressure is **global**
- Energy: temperature is advected and diffused; $\text{Pe} \approx 2,000$ means thin thermal layers
- Level set ϕ (dfun): signed distance to the bubble surface, carried by the flow

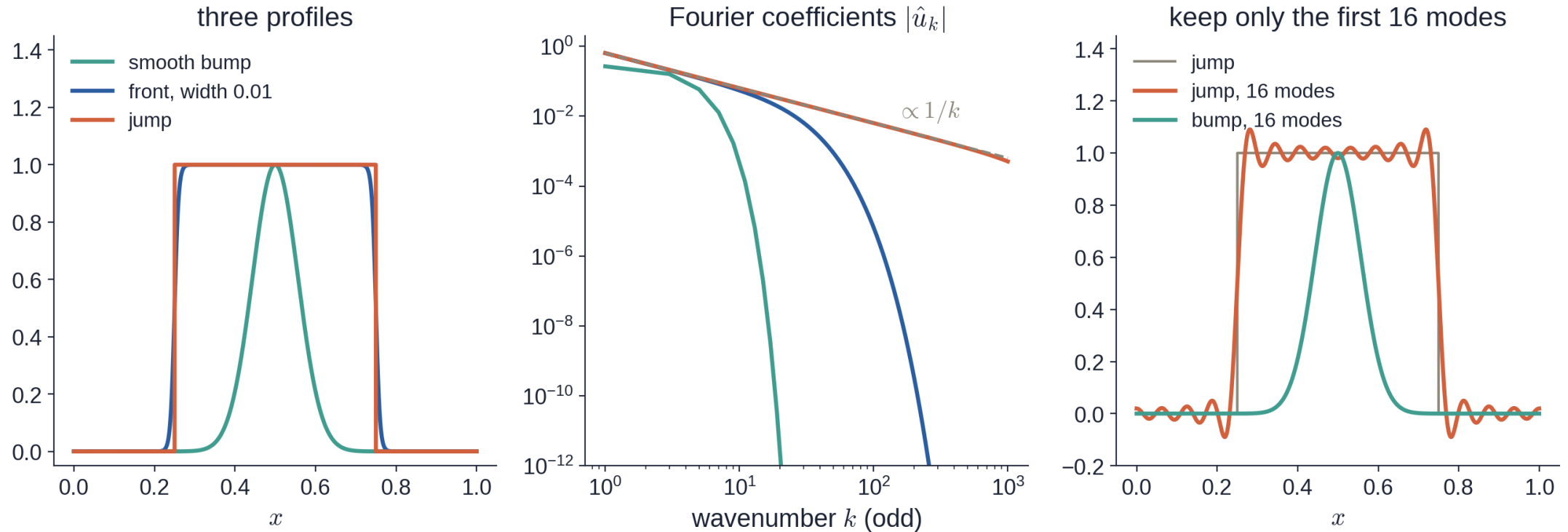


Smoothness means fast-decaying Fourier coefficients

$$u(x) = \sum_k \hat{u}_k e^{2\pi i k x}$$

- Any field on a periodic domain is a sum of waves. Smooth fields need few; sharp features need many
- Modes needed for 1% L^2 error
 - smooth bump: **8**; front of width 0.01: **35**; width 0.005: **61**; a jump: **over 1,600**
 - halving the front width nearly doubles the modes; a jump's coefficients fall only like $1/k$
- Keep a fixed number of modes: fine for smooth fields; a jump overshoots by **9%** (*Gibbs phenomenon*)
- Diffusion damps high modes; nonlinear advection creates them. FNO keeps a fixed number of low modes (Lecture 11): **a bubble interface is the hard case**

Sharp features need many modes



- Middle: the jump's coefficients follow $1/k$ forever; the front's follow it, then collapse at a wavenumber set by its width
- Right: with 16 modes the bump is exact to 4×10^{-9} ; the jump ripples and overshoots by 9%

A fixed number of Fourier modes blurs or ripples at an interface.

Measuring error: norms

- A field is a function, so an error is also a function. A **norm** measures its size
- **L²**: the root-mean-square error. On a uniform grid, the MSE loss from Lecture 2 is its discrete version
- **H¹**: also measures the error in the gradient (are the slopes right?)
- **Sup norm**: the largest pointwise error
- Names you will see in operator-learning papers
 - L²: functions with $\int u^2$ finite; H¹: also $\int |\nabla u|^2$ finite
 - Sobolev space H^s: Fourier coefficients decay fast enough that the sum on the right is finite (larger s means smoother)
 - Banach space: a space with a norm; Hilbert space: one with an inner product (L², H¹)

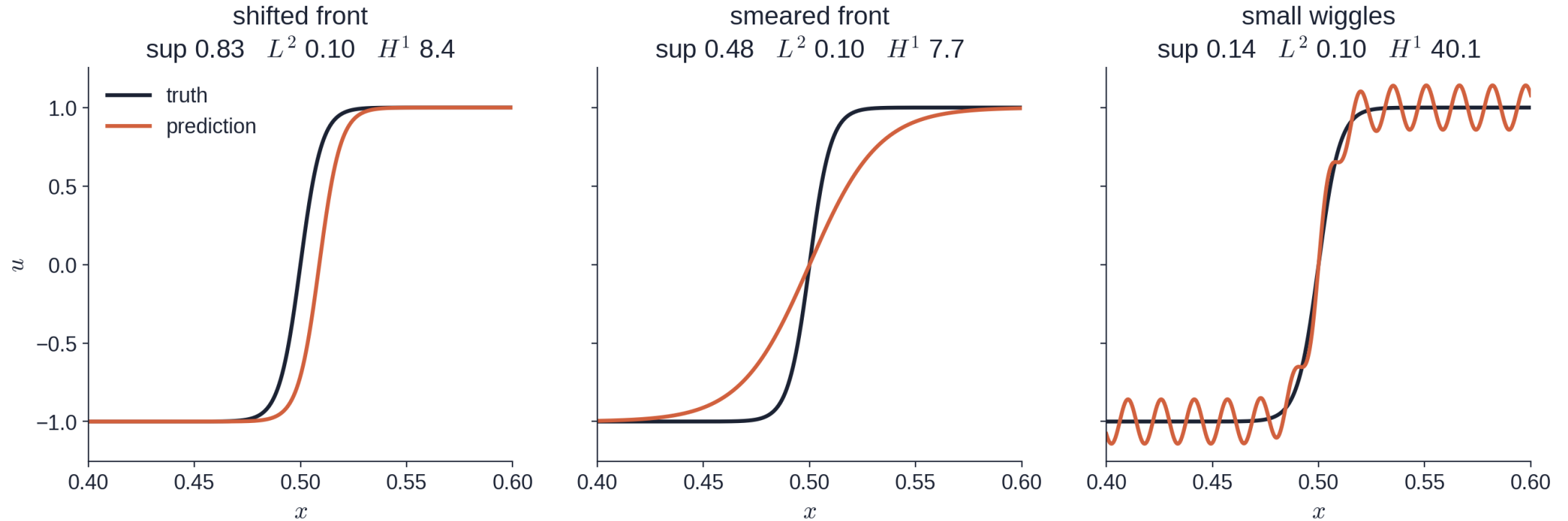
$$\|e\|_{L^2}^2 = \int e^2 dx \approx \frac{1}{N} \sum_i e_i^2 = \text{MSE}$$

$$\|e\|_{H^1}^2 = \int |\nabla e|^2 dx, \quad \|e\|_{\text{sup}} = \max_x |e(x)|$$

$$\|u\|_{H^s}^2 = \sum_k (1 + |k|^2)^s |\hat{u}_k|^2$$

MSE is the discrete L² norm. Other norms see other kinds of error.

Three predictions with the same MSE



- Truth: a sharp front of width 0.01 (a stand-in for a bubble interface). Each prediction has L^2 error **exactly 0.10**
- Shifted: large sup error. Smeared (*like our Lecture 1 rollout*): moderate on all three. Wiggles: small sup, large H^1

MSE **cannot tell** a shifted, a smeared and a noisy interface apart. *The science decides which matters.*

Finite differences from Taylor series

- Store u at grid points $x_i = i\Delta x$ and write $u_i \approx u(x_i)$; from here on $h = \Delta x$
- Expand the neighbors in a Taylor series, then subtract or add to isolate a derivative
- **Truncation error**: the Taylor terms we dropped. $O(h^2)$ means halving h cuts the error by **4**
- Other ways to discretize
 - **Finite volume**: store cell averages and update them with fluxes through the cell walls (slide 3); conserves *exactly*
 - **Finite element**: write u as a sum of basis functions, e.g. piecewise-linear "hat" functions; handles complex geometry
- This lecture uses finite differences: they map directly onto grids and CNNs

$$u(x \pm h) = u \pm h u' + \frac{h^2}{2} u'' \pm \frac{h^3}{6} u''' + \dots$$

$$\frac{u_{i+1} - u_i}{h} = u' + O(h) \quad \text{forward}$$

$$\frac{u_{i+1} - u_{i-1}}{2h} = u' + O(h^2) \quad \text{central}$$

$$\frac{u_{i+1} - 2u_i + u_{i-1}}{h^2} = u'' + O(h^2) \quad \text{second derivative}$$

Three ways to compute a derivative

$$f(x) = x^2 \sin x, \quad \text{find } f'(1)$$

Symbolic

differentiate the formula

$$\begin{aligned} f'(x) &= 2x \sin x + x^2 \cos x \\ f'(1) &= 2 \sin 1 + \cos 1 \\ &= 2.2232442755 \dots \end{aligned}$$

Exact. **The formula can grow huge** for a long program

Finite differences

evaluate f at nearby points

$\frac{f(1+h) - f(1-h)}{2h}$		
h	float64 error	float32 error
10^{-1}	4×10^{-3}	4×10^{-3}
10^{-3}	4×10^{-7}	9×10^{-6}
10^{-6}	3×10^{-11}	2×10^{-2}

Error depends on h ; **float32 hits round-off early**

Automatic differentiation

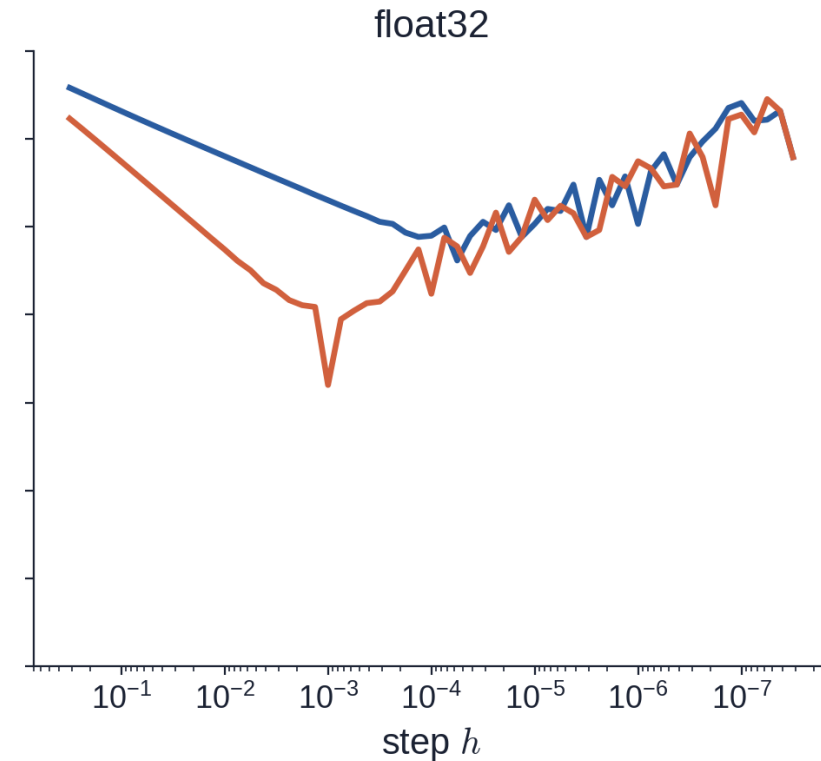
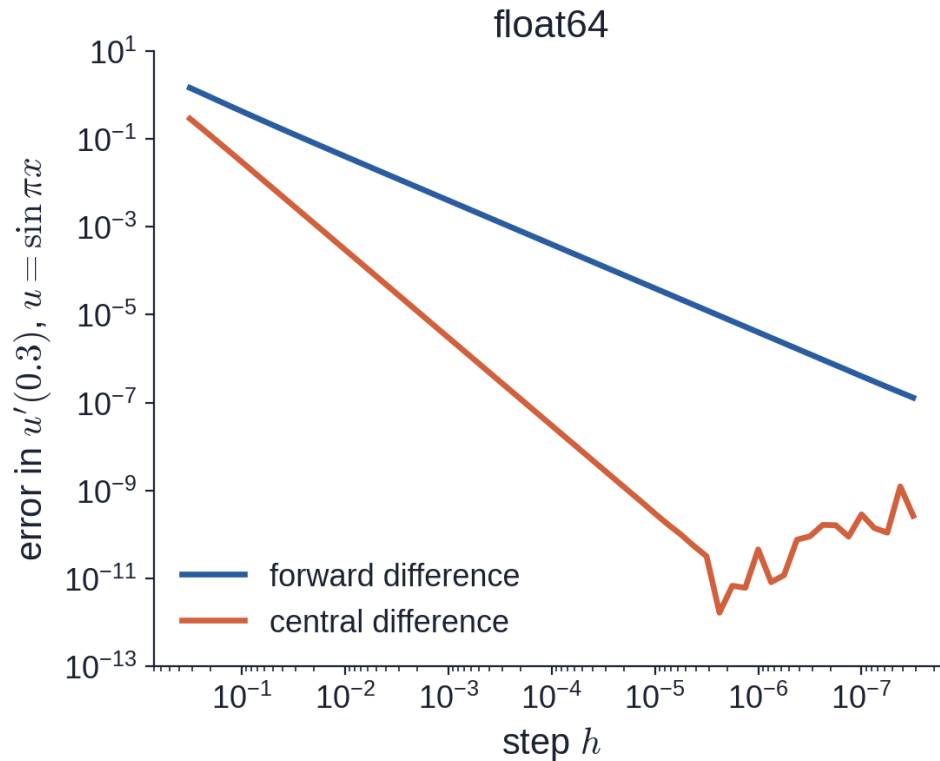
carry derivatives step by step

step	value	derivative
$v_1 = x$	1	$\dot{v}_1 = 1$
$v_2 = v_1^2$	1	$\dot{v}_2 = 2v_1 \dot{v}_1 = 2$
$v_3 = \sin v_1$	0.8415	$\dot{v}_3 = \cos v_1 \dot{v}_1 = 0.5403$
$v_4 = v_2 v_3$	0.8415	$\dot{v}_4 = \dot{v}_2 v_3 + v_2 \dot{v}_3 = 2.2232$

Exact up to rounding: error **0** (float64), **8×10^{-8}** (float32)

- Autodiff works on numbers: each step carries its value and its derivative (forward mode, above). Backpropagation runs the same chain rule backwards (reverse mode, Lecture 2)
- PINNs (Lecture 13) use it to get $\partial u / \partial x$ of a network with respect to its input

Round-off sets a floor



- float64: central differences reach about 10^{-11} , then round-off takes over
- float32 (the PyTorch default): best error about **10^{-6}** , near $h \approx 10^{-3}$; a smaller step gives **a worse derivative**

In float32, a smaller finite-difference step can give a worse derivative.

The Laplacian stencil is a convolution

$$\nabla^2 u_{i,j} \approx \frac{u_{i+1,j} + u_{i-1,j} + u_{i,j+1} + u_{i,j-1} - 4u_{i,j}}{\Delta x^2}$$

- Each new value combines a point with its four neighbors using fixed weights: *exactly* a 3×3 convolution
- The stencil's weights come from Taylor series. A CNN learns its weights, with many channels and nonlinearities in between
- Question: which 3×3 kernel approximates $\partial u / \partial x$?
- On an unstructured mesh the same idea becomes **message passing** between neighboring nodes

Finite-difference stencil SmallCNN kernel (Lecture 1)

0	1	0
1	-4	1
0	1	0

fixed; multiply by $1/\Delta x^2$

w_1	w_2	w_3
w_4	w_5	w_6
w_7	w_8	w_9

learned by gradient descent

A finite-difference solver is a CNN with hand-derived weights and no training.

Time stepping: the method of lines

- Discretize space first: the vector of grid values $\mathbf{u}$ obeys an ODE, where L is the matrix form of the stencil
- **Forward (explicit) Euler**: new state = old state + $\Delta t \times$ right-hand side
- Compare the Lecture 1 SmallCNN: **the same residual form**
 - new state = old state + a learned local update $f(\mathbf{u}; \theta)$
 - SmallCNN: four 3 x 3 convolutions with GELU, 9 x 9 receptive field
- **Backward (implicit) Euler**: the right-hand side uses the new state. Stable *for any* Δt , but every step solves a linear system
- Neural ODEs (Lecture 8) learn the right-hand side and pass it to an ODE solver

method of lines

$$\frac{d\mathbf{u}}{dt} = \alpha L \mathbf{u}$$

forward Euler

$$\mathbf{u}^{n+1} = \mathbf{u}^n + \Delta t \alpha L \mathbf{u}^n$$

SmallCNN (Lecture 1)

$$\mathbf{u}_{n+1} = \mathbf{u}_n + f(\mathbf{u}_n; \theta)$$

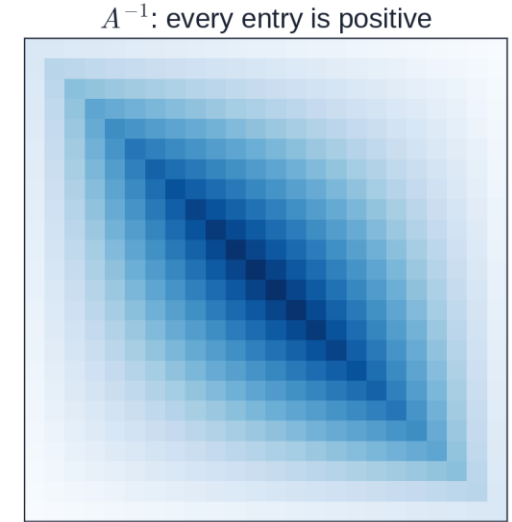
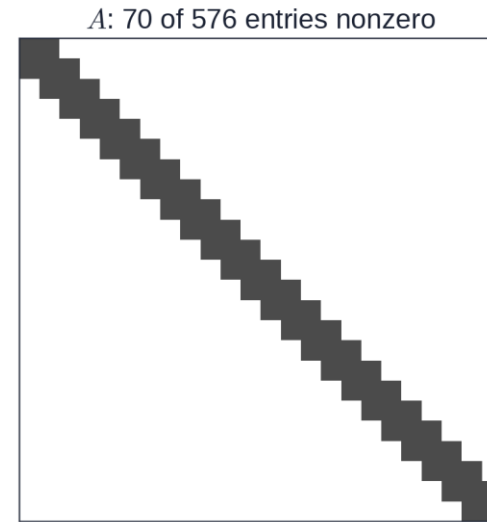
backward Euler

$$\mathbf{u}^{n+1} = \mathbf{u}^n + \Delta t \alpha L \mathbf{u}^{n+1}$$

Forward Euler and SmallCNN share one form: new state = old state + a local update.

Steady problems become linear systems

- Discretizing $-u'' = f$ gives $A u = b$. A is tridiagonal: each point talks to its two neighbors
- A^{-1} is **dense** and every entry is positive: the solution at each point depends on the source at every point (the elliptic case, as a matrix)
- Ways to solve: direct (sparse LU), iterative (Jacobi, conjugate gradients), multigrid
- **Multigrid**: smooth on the fine grid, correct on coarser grids, interpolate back up; cost grows *only linearly* with the number of unknowns
 - a U-Net has the same shape: downsample, process, upsample
- In incompressible solvers such as BubbleML's, the pressure Poisson solve is often the most expensive part of each step



$$-u'' = f \quad \longrightarrow \quad A \mathbf{u} = \mathbf{b}, \quad A = \frac{1}{\Delta x^2} \begin{pmatrix} 2 & -1 & & \\ -1 & 2 & -1 & \\ & \ddots & \ddots & \ddots \\ & & -1 & 2 \end{pmatrix}$$

A local operator with a global inverse: why elliptic problems need multigrid, or U-Nets.

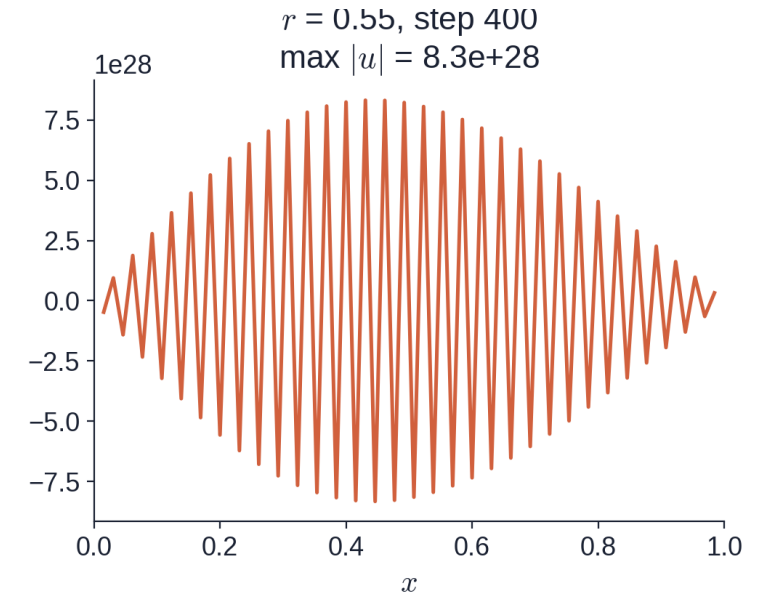
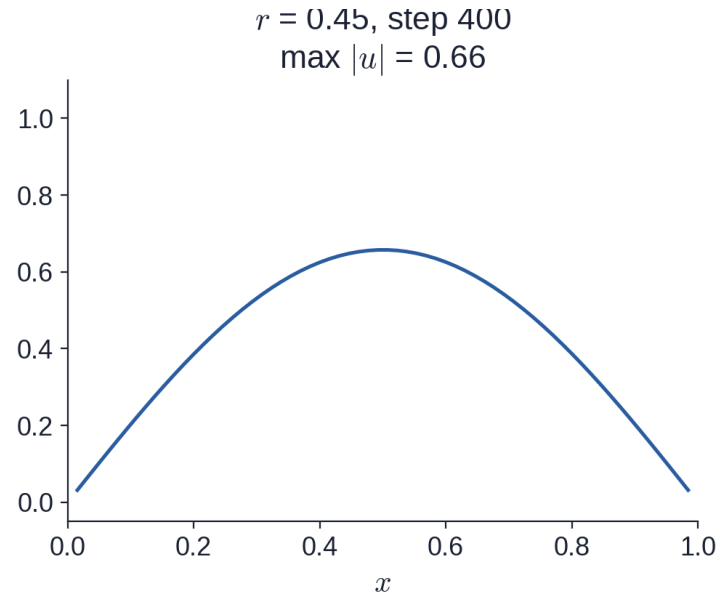
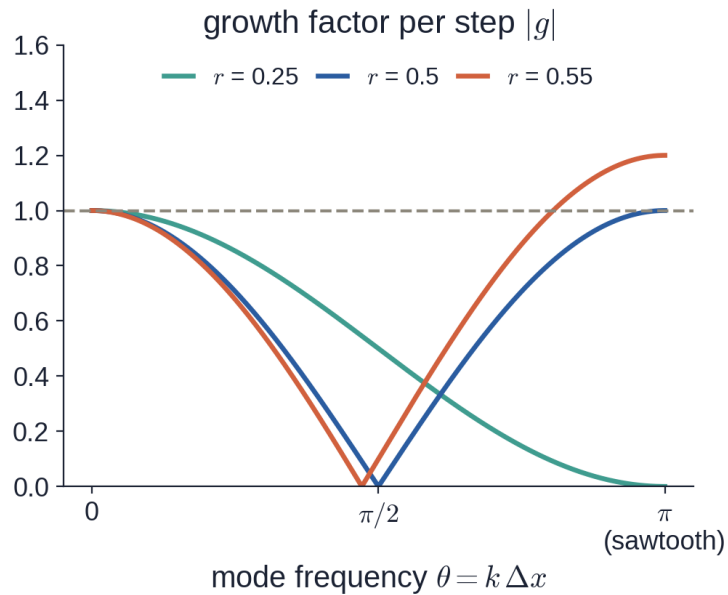
Stability

$$\hat{u}_k^{n+1} = g \hat{u}_k^n, \quad g(\theta) = 1 - 4r \sin^2 \frac{\theta}{2}, \quad r = \frac{\alpha \Delta t}{\Delta x^2}$$

- Feed one Fourier mode into forward Euler for the heat equation: each step multiplies it by g
 - $\theta = k\Delta x$: near 0 for smooth modes, π for the grid-scale sawtooth; r is the diffusion number
- **Stable** if $|g| \leq 1$ for every mode: $r \leq \frac{1}{2}$ in 1D, $r \leq \frac{1}{4}$ in 2D (von Neumann analysis)
- Advection has the **CFL condition** $c\Delta t/\Delta x \leq 1$: information may not travel more than one cell per step (Courant, Friedrichs, Lewy 1928)
- Backward Euler has $|g| \leq 1$ for every Δt : stable *at any step size*, at the cost of a linear solve

Stability limits the time step. A learned rollout can blow up for the same reason.

Stability: the same scheme at $r = 0.45$ and $r = 0.55$



Only Δt changed: at $r = 0.55$ the sawtooth grows $1.2\times$ per step, and noise of 0.01 becomes 10^{28} .

Consistency + stability = convergence

- **Local truncation error** τ : plug the exact solution into the discrete scheme; what is left over

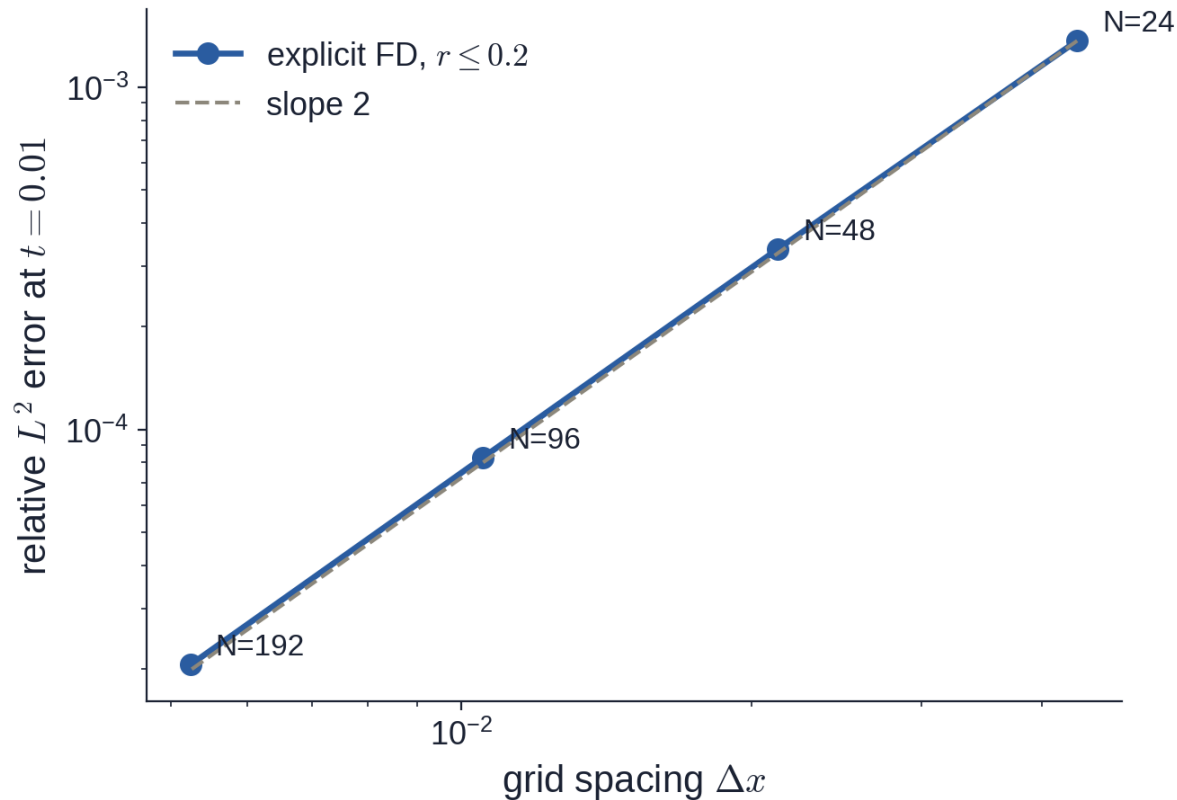
$$\tau = \frac{u(t + \Delta t) - u(t)}{\Delta t} - \alpha L u(t) = O(\Delta t) + O(\Delta x^2)$$

- **Consistent**: $\tau \rightarrow 0$ as $\Delta x, \Delta t \rightarrow 0$ (the scheme approximates the right PDE)
- **Stable**: errors are not amplified from step to step
- **Convergent**: the numerical solution approaches the exact one as $\Delta x, \Delta t \rightarrow 0$
- Lax equivalence theorem (Lax & Richtmyer 1956): for a well-posed linear problem, a consistent scheme converges if and only if it is stable

$$\text{consistent} + \text{stable} \iff \text{convergent}$$

Consistency alone is not enough: an unstable scheme does not converge.

Convergence on the Lecture 1 problem



- Explicit FD vs. the exact sine-series solution, from a Lecture 1-style initial condition
- Error drops by $\sim 4\times$ per halving of Δx : second order, as the Taylor analysis predicts
- Local error (one step) vs. global error (accumulated): like one-step loss vs. rollout error
- A network trained at one Δt has **no $\Delta t \rightarrow 0$ limit**, so the theorem gives no guarantee; its stability must be checked by rollout

Solvers come with a convergence guarantee. **Learned surrogates do not.**

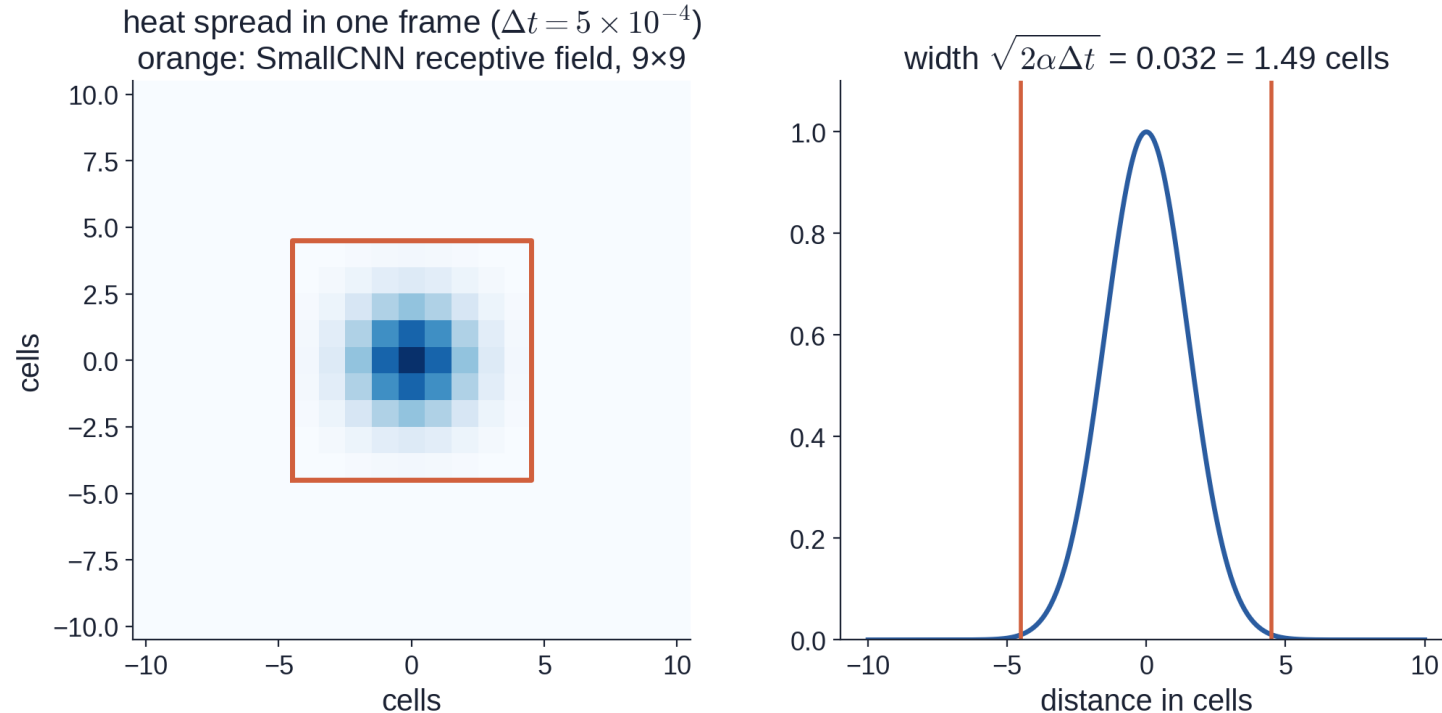
Checking the Lecture 1 data: the time step

$$r = \frac{\alpha \Delta t}{\Delta x^2} = \frac{1 \times 5 \times 10^{-4}}{(1/47)^2} \approx 1.10 > \frac{1}{4}$$

- Forward Euler needs $r \leq \frac{1}{4}$ in 2D: at least 5 substeps per saved frame
 - the CNN is asked to take a step that forward Euler **cannot take stably**
 - it learns a longer-range map than one solver step
- BubbleML: solver steps $\leq 4 \times 10^{-4}$, frames 1.0 apart, so thousands of solver steps per training pair
- Cost of explicit 3D diffusion: halving Δx means 8x the cells and 4x the steps, so **32x** the work
 - this is why fast learned surrogates are worth building

Our CNN takes one step where forward Euler would need five.

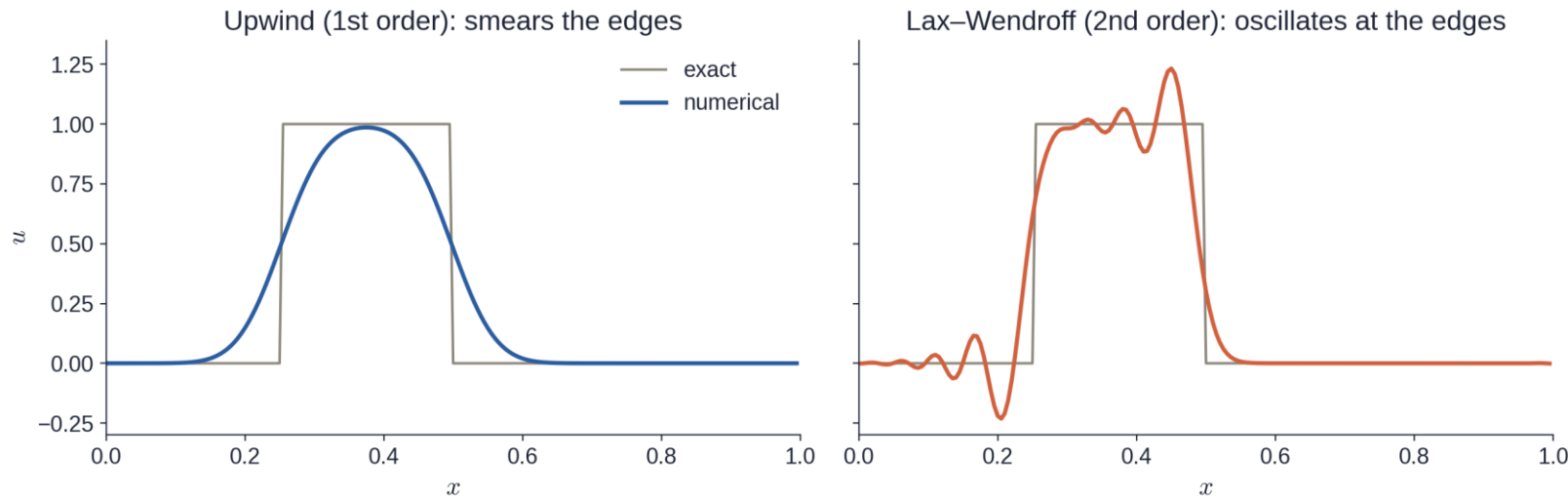
Checking the Lecture 1 data: can SmallCNN see far enough?



- Heat: one frame spreads a point over $\sqrt{(2\alpha\Delta t)} = 0.032$, about 1.5 cells, well inside the 9×9 receptive field. One reason the heat rollout stayed near 1% error
- At 96×96 the spread is 3 cells; at 192×192 it is 6, **beyond the 4-cell reach**. The same architecture would stop working on finer data
- BubbleML ($\Delta x = 0.25$): fluid at the 99th-percentile speed (1.9) moves about **8 cells** per frame, and the pressure is **global**

Numerical errors end up in the training data

- Advect a square wave once around a periodic domain (200 cells, $c\Delta t/\Delta x = 0.5$). The exact answer is the starting shape
 - **Upwind** (1st order): numerical diffusion *smears* the edges, as if an extra α had been added
 - **Lax-Wendroff** (2nd order): oscillations next to the edges; overshoot of **23%**
- Godunov (1959): a linear scheme that never creates new oscillations is at most first-order accurate. Modern solvers use nonlinear limiters to be sharp and non-oscillatory



Training data = PDE solution + discretization error. **A network learns both.**

What ML inherits from solvers

In a numerical solver	In ML for science
Local stencils	Convolutions (L6); message passing on meshes (L12)
Explicit time steps	Residual updates and autoregressive rollout (L7, L8)
Stability limits	Rollout blow-up; stability checked, not proved (L7)
Elliptic, global coupling	U-Nets (multigrid shape), attention, Fourier layers (L6, L9, L11)
Fourier modes and smoothness	FNO and its mode cutoff; blurring at sharp interfaces (L11)
Norms	Loss functions and evaluation metrics (L10)
Conservation, boundary conditions	Constraints a network ignores unless we build them in (L13, L14)
Fixed grid in the data	Resolution-independent operators (L11)
Chaos	Statistical evaluation, generative surrogates (L10, L15)
Discretization error	Part of the target; the network learns it too

Summary

- A PDE is a conservation law plus a flux law. With initial and boundary conditions it defines the solution operator $\mathcal{G}$
- Diffusion smooths, advection moves, nonlinear terms steepen and create new modes, elliptic problems couple the whole domain, some systems are chaotic
- Dimensionless numbers (Re, Pe) decide which behavior dominates
- Smooth fields have fast-decaying Fourier coefficients; sharp interfaces need many modes. The norm decides which errors count
- A finite-difference stencil is a convolution; an explicit time step is a residual update
- Consistency + stability = convergence. A learned surrogate has **no such guarantee**
- *Training data carries the solver's resolution, time step and errors*

After class

- 3Blue1Brown, "But what is a partial differential equation?" (Differential Equations, Ch. 2): the heat equation, visually
- LeVeque, Finite Difference Methods for Ordinary and Partial Differential Equations (SIAM, 2007)
 - Ch. 1 (finite differences), Ch. 9 (diffusion), Ch. 10 (advection)
- Thuerey et al., Physics-based Deep Learning (arXiv 2109.05237)
 - chapters on model equations and supervised training